PyMySQL
Chapter1
PyMySQL 介紹

Python 連線資料庫

1. **PyMySQL**（較為簡單，容易上手）
2. **MySQL Connector/Python**（MySQL 官方提供）
3. **SQLAlchemy**（ORM 框架，可搭配 PyMySQL / mysqlclient）
4. **asyncmy**（非同步 PyMySQL 替代品）
5. **mysqlclient**（原為 MySQLdb，需安裝 C 編譯器）

PyMySQL – 建立連線

```
import pymysql
# 建立 MySQL 資料庫連線
connection = pymysql.connect( # connect 函式回傳 Connection 物件
    host='localhost', # MySQL 伺服器的 IP 位址
    user='root',              # 登入 MySQL 的使用者名稱
    password='p@ssw0rd',      # 登入 MySQL 的密碼
    port=3306,                # MySQL 伺服器的端口
)
```

```
RuntimeError: 'cryptography' package is required for sha256_password or caching_sha2_password auth methods
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

註：如果出現該錯誤，需執行 `pip install cryptography`，並重啟 notebook

PyMySQL – cursor 常用的方法和屬性

方法/屬性	說明
`execute(query, args=None)`	執行單一 SQL 指令，支援參數化查詢以避免 **SQL injection**。
`executemany(query, args)`	一次執行多筆相同格式的 SQL 指令，常用於批次插入或更新。
`fetchone()`	取得查詢結果的下一筆資料（tuple），若無資料則回傳 None。
`fetchall()`	取得所有查詢結果資料（list of tuples），若無資料則回傳空 list。
`fetchmany(size)`	取得指定數量的資料列，回傳 list of tuples。
`close()`	關閉 cursor，釋放資源。
`rowcount`（屬性）	回傳最後一次執行 SQL 所影響的資料列數量（如 UPDATE, DELETE）。
`lastrowid`（屬性）	回傳最後一次 INSERT 操作所自動產生的 AUTO_INCREMENT 主鍵值。
`description`（屬性）	回傳欄位描述資訊（tuple of column info），可用來動態取得欄位名稱。

PyMySQL – 查詢資料庫

```
# 建立游標
cursor = connection.cursor()
# 使用游標執行 SQL 語句
cursor.execute("SHOW DATABASES;")
# 取得所有資料
dbs = cursor.fetchall()

print(dbs)

# 關閉游標
cursor.close()
```

PyMySQL – 建議的寫法

```
# 建立游標
with connection.cursor() as cursor:
    query = "SHOW DATABASES;"
    # 使用游標執行 SQL 語句
    cursor.execute(query)
    # 取得所有資料
    databases = cursor.fetchall()
```

離開 with 語句縮排時，自動關閉及釋放游標

PyMySQL – 查詢資料庫有哪些資料表

```
with connection.cursor() as cursor:
    # 使用游標執行 SQL 語句
    cursor.execute("SHOW TABLES FROM database_name;")
    # 取得所有資料
    tables = cursor.fetchall()

print(tables)
```

PyMySQL – 查詢資料

```
with connection.cursor() as cursor:
    # 使用 DB > TABLE 查詢
    sql = "SELECT * FROM database_name.table;"
    cursor.execute(sql)
    users = cursor.fetchall()

print(users)
```

PyMySQL – 建立連線(指定資料庫)

```
import pymysql
connection = pymysql.connect(
    host='localhost',
    user='root',
    password='p@ssw0rd',
    port=3306,
    database='database_name' # 指定資料庫
)
```

PyMySQL – 查詢資料

```python
with connection.cursor() as cursor:
    sql = "SELECT * FROM table;"
    cursor.execute(sql)
    users = cursor.fetchall()
print(users)
```

```
((1,
  'Admin',
  None,
  'AdminUser',
  'BioInf/IT',
  datetime.datetime(2025, 2, 8, 9, 51, 8, 434477)),
 (2,
  'jarvis_lu',
  '',
  'Jarvis Lu',
  'BioInf/IT',
  datetime.datetime(2025, 2, 19, 15, 43, 0, 610623)))
```

預設回傳的資料型別是 **tuple**

且沒有回傳欄位名稱

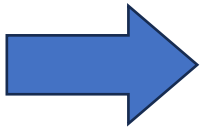
不方便後續處理使用

PyMySQL – 建立連線(修改回傳格式)

```
import pymysql
connection = pymysql.connect(
    host='localhost',
    user='root',
    password='p@ssw0rd',
    port=3306,
    database='database_name',
    cursorclass=pymysql.cursors.DictCursor # 查詢結果以字典形式返回
)
```

PyMySQL – 使用 DictCursor 後

```
((1,
  'Admin',
  None,
  'AdminUser',
  'BioInf/IT',
  datetime.datetime(2025, 2, 8, 9, 51, 8, 434477)),
 (2,
  'jarvis_lu',
  '',
  'Jarvis Lu',
  'BioInf/IT',
  datetime.datetime(2025, 2, 19, 15, 43, 0, 610623)),
 (3, 'Gene', None, 'Gene_Manager', 'BioInf/IT', None),
 (4,
  'Jason',
  None,
  'Jason',
  'BioInf/IT',
  datetime.datetime(2025, 2, 19, 15, 48, 39, 29619)))
```



```
[{'ch_name': None,
  'department': 'BioInf/IT',
  'id': 1,
  'last_login': datetime.datetime(2025, 2, 8, 9, 51, 8, 434477),
  'name': 'AdminUser',
  'username': 'Admin'},
 {'ch_name': '',
  'department': 'BioInf/IT',
  'id': 2,
  'last_login': datetime.datetime(2025, 2, 19, 15, 43, 0, 610623),
  'name': 'Jarvis Lu',
  'username': 'jarvis_lu'},
 {'ch_name': None,
  'department': 'BioInf/IT',
  'id': 3,
  'last_login': None,
  'name': 'Gene_Manager',
  'username': 'Gene'},
 {'ch_name': None,
  'department': 'BioInf/IT',
  'id': 4,
  'last_login': datetime.datetime(2025, 2, 19, 15, 48, 39, 29619),
  'name': 'Jason',
  'username': 'Jason'}]
```

configparser – 將帳號密碼寫在設定檔中

如果使用 **GitHub** 管理專案，不要將正式的帳號密碼寫在程式中。

config.ini

```
[DB]
host=localhost
user=user
password=password
port=port
```

configparser – 讀取設定檔

```
from configparser import ConfigParser

# 讀取 config.ini 檔案取得資料庫連線資訊
config = ConfigParser()
config.read('config.ini')

print(config['DB']) # 讀取 DB 區塊
print(config['DB']['host']) # 讀取 host
print(config['DB']['user']) # 讀取 user
```

configparser – 從設定檔建立資料庫連線

```
# 建立資料庫連線
connection = pymysql.connect(
    host=config.get('DB', 'host'),
    user=config.get('DB', 'user'),
    password=config.get('DB', 'password'),
    port=config.getint('DB', 'port'),
    cursorclass=pymysql.cursors.DictCursor,
)
```

PyMySQL – 將 SQL 查詢寫成函式

```
def sql_query(query):
    connection = pymysql.connect(
        host=config.get('DB', 'host'),
        user=config.get('DB', 'user'),
        password=config.get('DB', 'password'),
        port=config.getint('DB', 'port'),
        cursorclass=pymysql.cursors.DictCursor,
    )
    with connection.cursor() as cursor:
        cursor.execute(query) # 將 SQL 語句設為參數
        result = cursor.fetchall()
    return result
```

PyMySQL
Chapter2
INSERT / UPDATE

PyMySQL – 建立資料庫

```
with connection.cursor() as cursor:
    sql = """
        CREATE DATABASE IF NOT EXISTS chapter2;
    """
    # 執行建立的 SQL 語句
    cursor.execute(sql)
    # 執行查看資料庫
    cursor.execute("SHOW DATABASES;")
    result = cursor.fetchall()
print(result)
```

PyMySQL – 建立資料表

```
with connection.cursor() as cursor:
    sql = """
        CREATE TABLE IF NOT EXISTS users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(255) NOT NULL,
            age INT NOT NULL,
            username VARCHAR(255) NOT NULL,
            password VARCHAR(255) NOT NULL
        );
    """
    cursor.execute(sql)
```

PyMySQL – 寫入資料

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456")
    """
    # 執行寫入的 SQL 語句
    cursor.execute(sql)
    # 執行查詢資料表
    cursor.execute("SELECT * FROM users;")
    # 取得查詢的所有資料
    result = cursor.fetchall()
```

PyMySQL – connection.commit()

如果沒有執行 commit()，資料只會存在記憶體，關閉程式或連線後就會消失，不會寫進資料庫。所以只要是會改變**資料表內容**的操作，都需要在操作完成後呼叫一次 commit()以提交變更。

例如：

- **INSERT（新增資料）**
- **UPDATE（更新資料）**
- **DELETE（刪除資料）**

PyMySQL – 寫入資料

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456")
    """
    cursor.execute(sql)
    # 提交資料庫的變更
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – 更新資料

```
with connection.cursor() as cursor:
    sql = """
        UPDATE users SET age = 26, name = "Jarvis Lu" WHERE id = 1;
    """
    cursor.execute(sql)
    # 提交資料庫的變更
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – 使用格式化字串

```
with connection.cursor() as cursor:
    query = """
        SELECT * FROM main_user LIMIT %s
    """
    cursor.execute(query, (1,))
    result = cursor.fetchall()
```

SQL 語句參數僅支援 %s 佔位符

PyMySQL – SQL injection

```
username = "' OR 1=1; -- "
password = ""
with connection.cursor() as cursor:
    sql = f"""
    SELECT * FROM users WHERE username = '{username}' AND
    password = '{password}'
    """
    cursor.execute(sql)
    result = cursor.fetchall()
print(result)
```

勿使用 f-string 搭配 SQL 語句

PyMySQL – SQL injection

```
sql = f"""
SELECT * FROM users WHERE username = '{username}' AND
password = '{password}'
"""
```

如果 username 輸入 `' OR 1=1; --` ，使用 f-string 拼接會變成

```
SELECT * FROM users WHERE username = '' OR 1=1; -- ' AND password = ''
```

導致整個密碼條件失效。

而使用 `%s` 時，表示一個字串格式化占位符

會將整個參數視為一個完整的字串，無法從中終結SQL指令。

PyMySQL – 寫成函式

```
def insert_user(name, age, username, password):
    connection = pymysql.connect( …… )

    with connection.cursor() as cursor:
        sql = """
        INSERT INTO user (name, age, username, password)
        VALUES (%s, %s, %s, %s)
        """

        cursor.execute(sql, (name, age, username, password))
        connection.commit()
```

PyMySQL – executemany()

```
users_list = []
for i in range(101, 201):
    users_list.append((f'user_{i}', i + 20, f'user_{i}', f'password_{i}'))
    # [('user_101', 121, 'user_101', 'password_101'), ('user_102', 122, 'user_102', 'password_102'), ...]

with connection.cursor() as cursor:
    sql = """
    INSERT INTO user (name, age, username, password)
    VALUES (%s, %s, %s, %s);
    """
    cursor.executemany(sql, users_list)
    connection.commit()
```

PyMySQL
Chapter3
datetime 使用

建立含時間資訊的資料表

```
with connection.cursor() as cursor:
    sql = """
        CREATE TABLE IF NOT EXISTS users (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(255) NOT NULL,
            age INT NOT NULL,
            username VARCHAR(255) NOT NULL,
            password VARCHAR(255) NOT NULL,
            last_login TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        );
    """
    cursor.execute(sql)
```

寫入資料並查詢

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456");
    """
    cursor.execute(sql)
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – datetime

在使用 `cursor.fetchall()` 查詢 SQL 資料庫時，如果資料表中有 DATETIME 或 TIMESTAMP 類型的欄位，**PyMySQL** 會把這些欄位的值轉成 Python 的 `datetime.datetime` 物件。

```
[{'age': 25,
  'id': 1,
  'last_login': datetime.datetime(2025, 5, 25, 15, 52, 22),
  'name': 'Jarvis',
  'password': '123456',
  'username': 'jarvis'}]
```

datetime – 常用的方法

方法/屬性	說明	範例
`datetime.now()`	取得當前日期與時間	`datetime.now()`
`date.today()`	取得當前日期（無時間）	`date.today()`
`datetime(year, month, day, hour, min, sec)`	建立 datetime 物件	`datetime(2025, 5, 19, 14, 30)`
`dt.date()`	取得日期部分	`datetime.now().date()`
`dt.time()`	取得時間部分	`datetime.now().time()`
`dt + timedelta(days=1)`	日期加一天	`datetime.now() + timedelta(days=1)`
`dt1 > dt2`	比較兩個 datetime	`datetime(2025,1,1) > datetime(2024,1,1)`
`dt.strftime('%Y-%m-%d')`	轉為字串格式	`datetime.now().strftime('%Y/%m/%d')`
`datetime.strptime(str, fmt)`	字串轉 datetime	`datetime.strptime('2025-05-19', '%Y-%m-%d')`
`dt.weekday()`	傳回 0~6（週一～週日）	`datetime.now().weekday()`

datetime – 格式符號

格式符號	說明	範例
%Y	年（4 位數）	2025
%y	年（2 位數）	25
%m	月（01–12）	05
%d	日（01–31）	19
%H	小時（00–23）	14
%M	分（00–59）	30
%S	秒（00–59）	45
%A	星期幾（英文全名）	Monday
%a	星期幾（縮寫）	Mon
%p	AM / PM	AM

datetime – 使用範例

```python
from datetime import datetime

# 取得當前時間
print(datetime.now()) # 2025-05-19 16:11:15.427674

# 取得指定時間的 datetime 物件
print(datetime(2025, 5, 28, 16, 11, 15)) # 2025-05-28 16:11:15

# 將 datetime 輸出成指定格式字串
print(datetime.now().strftime('%Y-%m-%d %H:%M:%S')) # 2025-05-19 16:11:15

# 取得民國年
now = datetime.now()
print(f"民國{now.year - 1911}年{now.month}月{now.day}日") # 114年5月19日
```

PyMySQL – 寫入資料並查詢

```
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password)
        VALUES ("Jarvis", 25, "jarvis", "123456");
    """
    cursor.execute(sql)
    connection.commit()
    cursor.execute("SELECT * FROM users;")
    result = cursor.fetchall()
pprint(result)
```

PyMySQL – 寫入datetime資料

```
datetime_now = datetime.now()
with connection.cursor() as cursor:
    sql = """
        INSERT INTO users (name, age, username, password, start_date, last_login)
        VALUES ("Jarvis", 25, "jarvis", "123456", %s, %s);
    """
    cursor.execute(sql, ("2025-05-24", datetime_now))
    connection.commit()
```

可使用
1. 標準 SQL 日期格式 'YYYY-MM-DD HH:MM:SS'
2. datetime 物件

PyMySQL – 更新datetime資料

```
from datetime import datetime, timedelta
yesterday = datetime.now() - timedelta(days=1)
with connection.cursor() as cursor:
    sql = """
        UPDATE users2 SET start_date = %s, last_login = %s
        WHERE id = %s;
    """
    cursor.execute(sql, (yesterday, "2025-05-28 10:30:50", 1))
    connection.commit()
    cursor.execute("SELECT * FROM users2;")
    result = cursor.fetchall()
```

PyMySQL
Chapter4
實例練習

零售超市的銷售記錄

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
1	CA-2016-1	11/8/2016	11/11/2016	Second Cla	CG-12520	Claire Gute	Consumer	FUR-BO-10	Furniture	Bookcases	Bush Somerset	261.96	2	0	41.9136
2	CA-2016-1	11/8/2016	11/11/2016	Second Cla	CG-12520	Claire Gute	Consumer	FUR-CH-10	Furniture	Chairs	Hon Deluxe Fa	731.94	3	0	219.582
3	CA-2016-1	6/12/2016	6/16/2016	Second Cla	DV-13045	Darrin Van Huff	Corporate	OFF-LA-10	Office Supj	Labels	Self-Adhesive .	14.62	2	0	6.8714
4	US-2015-1	10/11/2015	10/18/2015	Standard C	SO-20335	Sean O'Donnell	Consumer	FUR-TA-10	Furniture	Tables	Bretford CR45	957.5775	5	0.45	-383.031
5	US-2015-1	10/11/2015	10/18/2015	Standard C	SO-20335	Sean O'Donnell	Consumer	OFF-ST-10	Office Supj	Storage	Eldon Fold 'N I	22.368	2	0.2	2.5164
6	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	FUR-FU-10	Furniture	Furnishings	Eldon Expressi	48.86	7	0	14.1694
7	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	OFF-AR-10	Office Supj	Art	Newell 322	7.28	4	0	1.9656
8	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	TEC-PH-10	Technolog	Phones	Mitel 5320 IP I	907.152	6	0.2	90.7152
9	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	OFF-BI-10	Office Supj	Binders	DXL Angle-Vi	18.504	3	0.2	5.7825
10	CA-2014-1	6/9/2014	6/14/2014	Standard C	BH-11710	Brosina Hoffmar	Consumer	OFF-AP-10	Office Supj	Appliances	Belkin F5C206	114.9	5	0	34.47

將 CSV 資料切分

Customers

F	G	H
Customer ID	Customer Name	Segment
CG-12520	Claire Gute	Consumer
CG-12520	Claire Gute	Consumer
DV-13045	Darrin Van Huff	Corporate
SO-20335	Sean O'Donnell	Consumer
SO-20335	Sean O'Donnell	Consumer
BH-11710	Brosina Hoffmar	Consumer
BH-11710	Brosina Hoffmar	Consumer
BH-11710	Brosina Hoffmar	Consumer
BH-11710	Brosina Hoffmar	Consumer

Orders

B	C	D	E
Order ID	Order Date	Ship Date	Ship Mode
CA-2016-1	11/8/2016	11/11/2016	Second Cla
CA-2016-1	11/8/2016	11/11/2016	Second Cla
CA-2016-1	6/12/2016	6/16/2016	Second Cla
US-2015-1	10/11/2015	10/18/2015	Standard C
US-2015-1	10/11/2015	10/18/2015	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C
CA-2014-1	6/9/2014	6/14/2014	Standard C

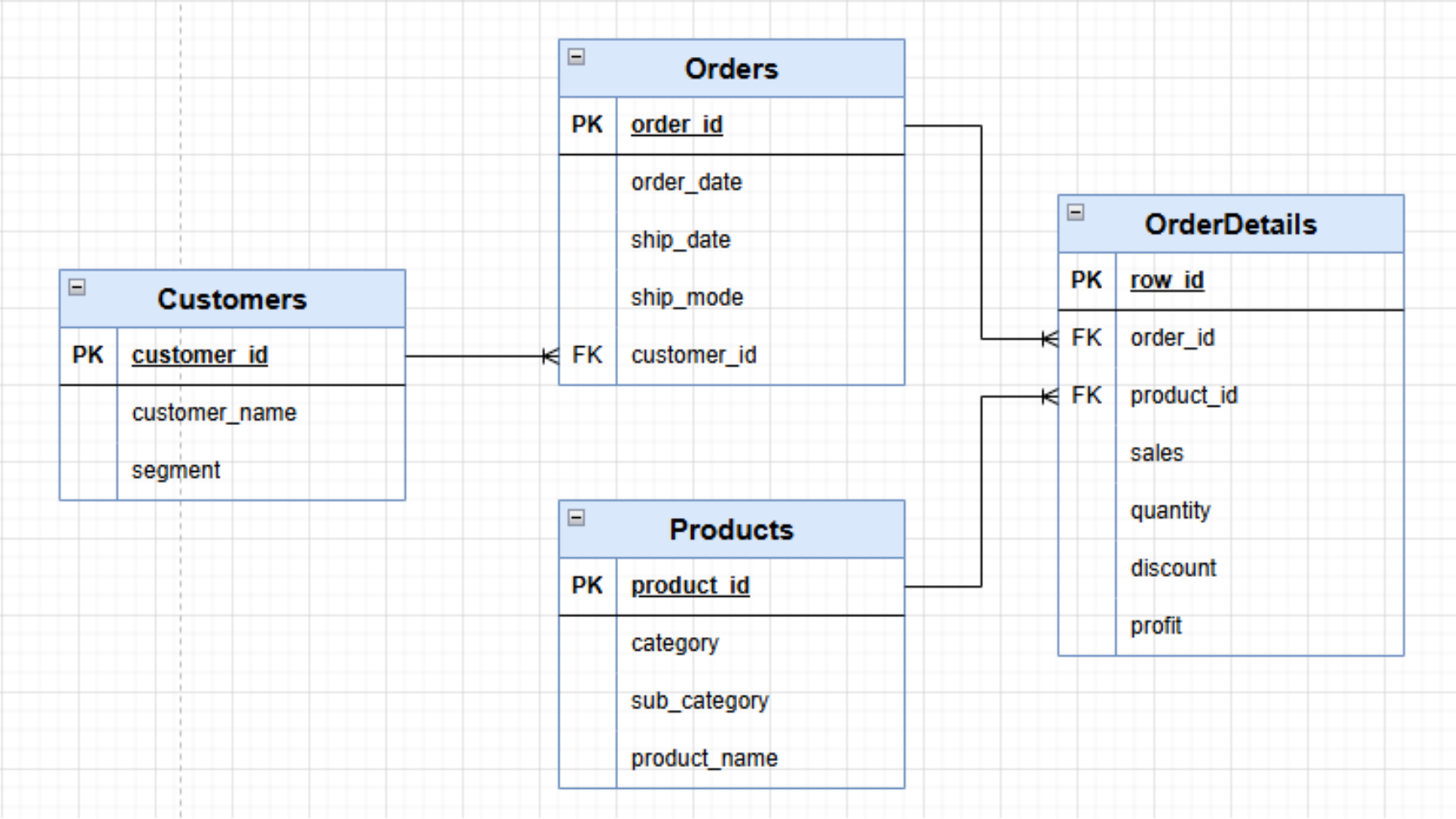
Products

I	J	K	L
Product ID	Category	Sub-Category	Product Name
FUR-BO-1C	Furniture	Bookcases	Bush Somerset
FUR-CH-1C	Furniture	Chairs	Hon Deluxe Fa
OFF-LA-10	Office Supj	Labels	Self-Adhesive
FUR-TA-10	Furniture	Tables	Bretford CR45
OFF-ST-10C	Office Supj	Storage	Eldon Fold 'N
FUR-FU-10	Furniture	Furnishings	Eldon Expressi
OFF-AR-10	Office Supj	Art	Newell 322
TEC-PH-10	Technology	Phones	Mitel 5320 IP
OFF-BI-10C	Office Supj	Binders	DXL Angle-Vi

OrderDetails

M	N	O	P
Sales	Quantity	Discount	Profit
261.96	2	0	41.9136
731.94	3	0	219.582
14.62	2	0	6.8714
957.5775	5	0.45	-383.031
22.368	2	0.2	2.5164
48.86	7	0	14.1694
7.28	4	0	1.9656
907.152	6	0.2	90.7152
18.504	3	0.2	5.7825

將 csv 資料切分


Orders
PK
order_id
order_date
ship_date
ship_mode
FK
customer_id
Customers
PK
customer_id
customer_name
segment
OrderDetails
PK
row_id
FK
order_id
FK
product_id
sales
quantity
discount
profit
Products
PK
product_id
category
sub_category
product_name

建立資料表

Customers

```
sql = """
    CREATE TABLE IF NOT EXISTS Customers (
        customer_id VARCHAR(20) PRIMARY KEY,
        customer_name VARCHAR(100),
        segment VARCHAR(50)
    );
"""
```

Orders

```
sql = """
    CREATE TABLE IF NOT EXISTS Orders (
        order_id VARCHAR(20) PRIMARY KEY,
        order_date DATE,
        ship_date DATE,
        ship_mode VARCHAR(50),
        customer_id VARCHAR(20),
        FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
    );
"""
```

建立資料表

Products

```
sql = """
    CREATE TABLE IF NOT EXISTS Products (
        product_id VARCHAR(20) PRIMARY KEY,
        category VARCHAR(50),
        sub_category VARCHAR(50),
        product_name VARCHAR(255)
    );
"""
```

OrderDetails

```
sql = """
    CREATE TABLE IF NOT EXISTS OrderDetails (
        row_id INT PRIMARY KEY,
        order_id VARCHAR(20),
        product_id VARCHAR(20),
        sales DECIMAL(10,2),
        quantity INT,
        discount DECIMAL(4,2),
        profit DECIMAL(10,2),
        FOREIGN KEY (order_id) REFERENCES Orders(order_id),
        FOREIGN KEY (product_id) REFERENCES Products(product_id)
    );
"""
```

讀取 csv 資料 – 提取特定欄位

```
import pandas as pd

# 讀取 CSV 檔案
df = pd.read_csv("Sample-Superstore.csv", encoding='latin1')
# 提取客戶欄位的資料
customers = df[['Customer ID', 'Customer Name', 'Segment']]
customers
```

	Customer ID	Customer Name	Segment
0	CG-12520	Claire Gute	Consumer
1	CG-12520	Claire Gute	Consumer
2	DV-13045	Darrin Van Huff	Corporate
3	SO-20335	Sean O'Donnell	Consumer
4	SO-20335	Sean O'Donnell	Consumer
...	...	...	...
9989	TB-21400	Tom Boeckenhauer	Consumer
9990	DB-13060	Dave Brooks	Consumer
9991	DB-13060	Dave Brooks	Consumer
9992	DB-13060	Dave Brooks	Consumer
9993	CC-12220	Chris Cortes	Consumer

9994 rows × 3 columns

讀取 csv 資料 – 去除重複數據

```python
import pandas as pd

# 讀取 CSV 檔案
df = pd.read_csv("Sample-Superstore.csv", encoding='latin1')
# 提取客戶欄位的資料
customers = df[['Customer ID', 'Customer Name', 'Segment']]
# 去除重複的客戶資料
customers = customers.drop_duplicates()
customers
```

	Customer ID	Customer Name	Segment
0	CG-12520	Claire Gute	Consumer
2	DV-13045	Darrin Van Huff	Corporate
3	SO-20335	Sean O'Donnell	Consumer
5	BH-11710	Brosina Hoffman	Consumer
12	AA-10480	Andrew Allen	Consumer
...	...	...	...
8666	CJ-11875	Carl Jackson	Corporate
9209	RS-19870	Roy Skaria	Home Office
9399	SC-20845	Sung Chung	Consumer
9441	RE-19405	Ricardo Emerson	Consumer
9558	SM-20905	Susan MacKendrick	Consumer

793 rows × 3 columns

讀取 csv 資料 – 寫入資料庫

```
# 取得 df 中的客戶資料，並去除重複的資料
customers = df[['Customer ID', 'Customer Name', 'Segment']].drop_duplicates()
# 將客戶資料轉換為 list
customers_value_list = customers.values.tolist()

# 將資料寫入資料庫
with connection.cursor() as cursor:
    sql = """
    INSERT INTO Customers (customer_id, customer_name, segment) VALUES (%s, %s, %s)
    """
    cursor.executemany(sql, customers_value_list)
    connection.commit()
```

讀取 csv 資料 – 處理日期格式資料

```
# 取得 df 中的訂單資料
orders = df[['Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID']].drop_duplicates()
orders
```

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID
0	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520
2	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045
3	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335
5	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710
12	CA-2017-114412	4/15/2017	4/20/2017	Standard Class	AA-10480
...	...	...	...	...	...
9986	CA-2016-125794	9/29/2016	10/3/2016	Standard Class	ML-17410
9987	CA-2017-163629	11/17/2017	11/21/2017	Standard Class	RA-19885
9989	CA-2014-110422	1/21/2014	1/23/2014	Second Class	TB-21400
9990	CA-2017-121258	2/26/2017	3/3/2017	Standard Class	DB-13060
9993	CA-2017-119914	5/4/2017	5/9/2017	Second Class	CC-12220

5009 rows × 5 columns

讀取 csv 資料 – 處理日期格式資料

```
# 取得 df 中的訂單資料
orders = df[['Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID']].drop_duplicates()
# 將日期格式轉換為 datetime.date
orders['Order Date'] = pd.to_datetime(orders['Order Date'], format='%m/%d/%Y').dt.date
orders['Ship Date'] = pd.to_datetime(orders['Ship Date'], format='%m/%d/%Y').dt.date
orders
```

	Order ID	Order Date	Ship Date	Ship Mode	Customer ID
0	CA-2016-152156	2016-11-08	2016-11-11	Second Class	CG-12520
2	CA-2016-138688	2016-06-12	2016-06-16	Second Class	DV-13045
3	US-2015-108966	2015-10-11	2015-10-18	Standard Class	SO-20335
5	CA-2014-115812	2014-06-09	2014-06-14	Standard Class	BH-11710
12	CA-2017-114412	2017-04-15	2017-04-20	Standard Class	AA-10480
...	...	...	...	...	...
9986	CA-2016-125794	2016-09-29	2016-10-03	Standard Class	ML-17410
9987	CA-2017-163629	2017-11-17	2017-11-21	Standard Class	RA-19885
9989	CA-2014-110422	2014-01-21	2014-01-23	Second Class	TB-21400
9990	CA-2017-121258	2017-02-26	2017-03-03	Standard Class	DB-13060
9993	CA-2017-119914	2017-05-04	2017-05-09	Second Class	CC-12220

5009 rows × 5 columns